

UCL DEPARTMENT OF COMPUTER SCIENCE

COMP0264: Generative Artificial Intelligence

Master Past Exam Revision Guide & Exhaustive Solutions

Compiling Examinations and Core Concepts for Cohorts 2025–2026

Prepared by **Antigravity AI** for UCL Computer Science

July 2026

Master Course Syllabus & Core Roadmap

This master revision guide compiles the examination papers and core theoretical foundations for UCL COMP0264 (Generative Artificial Intelligence):

- 2025–2026 Mock Examination Paper (Variant A):** Encoder/Decoder VAEs, Posterior Collapse, Transformers, LLMs, and Diffusion Models.
- 2025–2026 Mock Examination Paper (Variant B):** Generator/Discriminator VAEs vs GANs, Mode Collapse, and Scaling Laws.

The course covers four primary pillars of modern deep generative modeling:

Topic Area	Core Concepts and Key Formulations
1. VAEs & Latent Models	Encoder $q_\phi(z x)$, Decoder $p_\theta(x z)$, Reparameterization Trick, ELBO loss, Posterior Collapse, β -VAE.
2. GANs & Adversarial Models	Generator G_θ , Discriminator D_ϕ , Minimax game $V(G, D)$, JS-divergence, Mode Collapse, WGAN-GP.
3. Transformers & LLMs	Self-Attention, Scaled Dot-Product $\frac{QK^T}{\sqrt{d_k}}$, Positional Encoding (Sinusoidal, RoPE), Masking, Emergent ICL, RLHF/DPO.
4. Diffusion Models	Forward Process $q(x_t x_0)$, Reverse Process $p_\theta(x_{t-1} x_t)$, Reparameterization $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$, Denoising Loss.
Appendix	Standard Formulas: Chain Rule ($N = 2, 3, \dots, n$), Bayes' Rule, ELBO, Gaussian KL, Softmax Scaling, DPO Loss, Score Matching SDEs.

PART 1: COMPREHENSIVE MOCK EXAMINATION PAPERS

PAPER 1: COMP0264 Mock Examination 2025–2026

Duration: 3 Hours — Total Marks: 100

Question 1: Variational Autoencoders & GANs [25 Marks]

- (a) Describe the architecture of the encoder and decoder of a Variational Autoencoder (VAE). Discuss the training procedure including the loss function (ELBO) used for optimization. [12.5 Marks]
- (b) Explain why training VAEs can be unstable and discuss the causes of the potential problem of posterior collapse (i.e. the situation in which the encoder's output becomes almost identical to the prior, regardless of the input). Compare this to mode collapse in GANs and discuss possible solutions. [12.5 Marks]

Question 2: Transformer Architectures [25 Marks]

- (a) Explain the self-attention mechanism in transformer architectures and why it is effective compared to recurrent neural networks. [5 Marks]
- (b) Why is positional encoding necessary in transformers? Describe one method for incorporating positional information (e.g. sinusoidal or RoPE). [10 Marks]
- (c) Discuss why we need scaled self-attention. Provide a mathematical justification by deriving the variance scaling factor $\sqrt{d_k}$. [10 Marks]

Question 3: Large Language Models (LLMs) [25 Marks]

- (a) Despite being trained with a next-token prediction objective, LLMs exhibit emergent behaviors such as in-context learning (ICL). Provide a critical explanation, grounded in probabilistic modeling, of why such behavior arises. [5 Marks]
- (b) In transformer-based LLMs, the conditional distribution $p(x_t | x_{<t})$ is parameterized using masked self-attention. Explain how masking ensures the autoregressive property is preserved during training. [5 Marks]
- (c) Suppose an LLM assigns high probability to fluent but factually incorrect sequences. Explain why maximum likelihood training does not guarantee factual correctness, and discuss one potential solution (e.g. RLHF/DPO or RAG). [7.5 Marks]
- (d) Explain how fine-tuning can lead to catastrophic forgetting from a probabilistic perspective. Use the Kullback-Leibler (KL) divergence to formalize the intuition. [7.5 Marks]

Question 4: Diffusion Models [25 Marks]

- (a) Describe the forward and reverse processes in a diffusion model. What is the goal of each process? Formulate the underlying mathematical model for both processes and explain how the reverse process is derived from the forward process. [12.5 Marks]
- (b) How is the reverse decoder in a diffusion model trained? What computational challenges arise in doing so, and what practical approach (ϵ -prediction MSE objective) is used instead? [12.5 Marks]

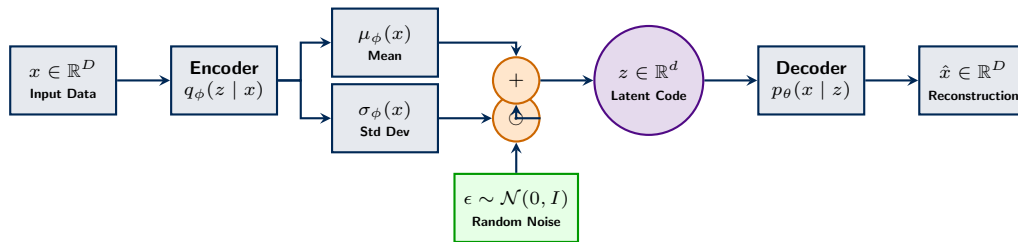
PART 2: EXHAUSTIVE MASTER SOLUTIONS GUIDE

Solution 1: Variational Autoencoders & GANs

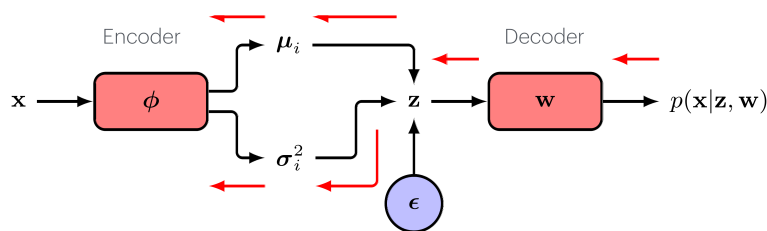
Part (a): VAE Architecture, Reparameterization Trick & ELBO Derivation

Core Concept Summary: VAEs optimize the Evidence Lower Bound (ELBO) using an encoder $q_\phi(z | x)$ and decoder $p_\theta(x | z)$. The Reparameterization Trick ($z = \mu + \sigma \odot \epsilon$) enables end-to-end gradient backpropagation through stochastic latent bottlenecks.

Perspective 1: Vector Computational Graph & Reparameterization Trick (TikZ Block Diagram)



The Reparameterisation Trick



Generative AI 2025-26

Mirco Musolesi

Figure 1: *Official Lecture Diagram: VAE Architecture & Reparameterization Trick.*

Perspective 2: Official Lecture Slide Architecture (COMP0264 Lecture 4, Slide 36)

1. Encoder and Decoder Architecture: A VAE consists of two neural networks:

- **Encoder** $q_\phi(z | x)$: Maps high-dimensional input $x \in \mathbb{R}^D$ to the parameters of a continuous latent distribution (typically a Gaussian $\mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$). Outputs mean $\mu_\phi(x)$ and log-variance $\log \sigma_\phi^2(x)$.
- **Decoder** $p_\theta(x | z)$: Maps latent code $z \in \mathbb{R}^d$ ($d \ll D$) back to data space, parameterizing distribution $p_\theta(x | z)$ (e.g. Bernoulli for binary data or Gaussian $\mathcal{N}(\mu_\theta(z), \sigma^2 I)$ for continuous data).

2. The Reparameterization Trick: Direct sampling $z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$ is non-differentiable with respect to encoder parameters ϕ . The reparameterization trick isolates stochasticity by sampling an independent noise vector $\epsilon \sim \mathcal{N}(0, I)$ and expressing z deterministically as:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

This allows backpropagation of gradients ∇_ϕ via standard chain rule through μ_ϕ and σ_ϕ .

3. **Mathematical Derivation of the Evidence Lower Bound (ELBO):** We wish to maximize true data log-likelihood $\log p_\theta(x) = \log \int p_\theta(x, z) dz$. Introducing variational posterior $q_\phi(z | x)$:

$$\log p_\theta(x) = \log \int q_\phi(z | x) \frac{p_\theta(x, z)}{q_\phi(z | x)} dz = \log \mathbb{E}_{q_\phi(z | x)} \left[\frac{p_\theta(x, z)}{q_\phi(z | x)} \right]$$

Applying **Jensen's Inequality** ($\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$ since $\log$ is concave):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z | x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] \equiv \mathcal{L}_{\text{ELBO}}(\theta, \phi)$$

Expanding $\mathcal{L}_{\text{ELBO}}$:

$$\begin{aligned} \mathcal{L}_{\text{ELBO}}(\theta, \phi) &= \mathbb{E}_{q_\phi(z | x)} \left[\log \frac{p_\theta(x | z) p(z)}{q_\phi(z | x)} \right] \\ &= \mathbb{E}_{q_\phi(z | x)} [\log p_\theta(x | z)] - \mathbb{E}_{q_\phi(z | x)} \left[\log \frac{q_\phi(z | x)}{p(z)} \right] \\ &= \underbrace{\mathbb{E}_{q_\phi(z | x)} [\log p_\theta(x | z)]}_{\text{Reconstruction Loss}} - \underbrace{KL(q_\phi(z | x) \parallel p(z))}_{\text{KL Regularization Term}} \end{aligned}$$

For Gaussian prior $p(z) = \mathcal{N}(0, I)$ and encoder $q_\phi(z | x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$, the KL divergence integrates analytically:

$$KL(q_\phi(z | x) \parallel p(z)) = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Part (b): Posterior Collapse in VAEs vs Mode Collapse in GANs

Core Concept Summary: Posterior collapse in VAEs occurs when powerful decoders ignore latent codes ($KL \rightarrow 0$), whereas Mode Collapse in GANs occurs when generators collapse to producing samples from only a single data mode.

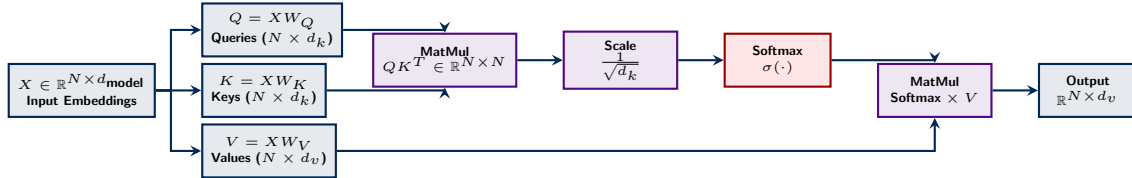
- Posterior Collapse in VAEs:** Posterior collapse occurs when the variational posterior matches the prior independent of input: $q_\phi(z | x) \approx p(z) = \mathcal{N}(0, I)$, causing $KL(q_\phi(z | x) \parallel p(z)) \rightarrow 0$. The encoder completely ignores input x , and the decoder ignores latent z .
 - Cause:** Occurs when using an overly powerful autoregressive decoder (e.g. PixelCNN or Transformer). A strong decoder can model $p(x)$ directly without using z . Since $KL \rightarrow 0$ maximizes the ELBO objective, the optimizer chooses the degenerate path of setting $q(z|x) = p(z)$.
 - Solutions:**
 - β -VAE Annealing:** Multiply KL term by weight β . Gradually increase β from 0 to 1 during training (β schedule), forcing the model to learn meaningful z early when $\beta = 0$.
 - Free Bits / KL Minimum Margin:** Enforce a lower bound threshold λ on individual latent dimensions: $\max(\lambda, KL(q_\phi(z_i|x) \parallel p(z_i)))$.
 - VQ-VAE (Vector Quantized VAE):** Replace continuous Gaussian latent space with discrete codebook vectors, preventing KL collapse.
- Mode Collapse in GANs:** In GANs ($\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$), mode collapse occurs when generator G learns to produce samples from only a single mode or narrow subset of data modes (e.g. generating only 1 digit out of 10 in MNIST).
 - Cause:** GAN objective optimizes Jensen-Shannon divergence $JS(p_{\text{data}} \parallel p_g)$ when D is optimal. JS divergence is zero-forcing for missing modes, allowing G to over-optimize on a single real-looking mode that fools D .
 - Solutions:** Wasserstein GAN with Gradient Penalty (WGAN-GP) using Earth Mover's distance, spectral normalization of D , and unrolled GANs.

Solution 2: Transformer Architectures

Part (a): Self-Attention Mechanism

Core Concept Summary: Self-Attention computes pairwise token interaction matrices $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ in $O(1)$ path length, enabling parallel GPU training unlike sequential $O(N)$ RNN paths.

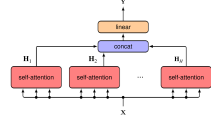
Perspective 1: Tensor Signal-Flow Diagram (TikZ Block Diagram)



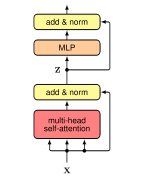
Multi-Head Attention

Transformer Layer and Transformer Architecture

Starting, from the definition of the (single) attention head introduced before:
 $Y = \text{Attention}(Q, K, V) = \text{softmax}(QK^T)V$
 We can now introduce a generic attention head H_k :
 $H_k = \text{Attention}(Q_k, K_k, V_k) = \text{softmax}(Q_k K_k^T) V_k$
 We had different query, key and value matrices for each head:
 $Q_k = XW_k^{(Q)}$
 $K_k = XW_k^{(K)}$
 $V_k = XW_k^{(V)}$



This architecture can be further improved by adding another residual connection (with layer normalisation).
 In formal way:
 $\tilde{X} = \text{LayerNorm}[\text{MLP}[Z] + Z]$
 This overall composition is usually referred to as *transformer layer*.
 Typically, we then stack several transformation layers.
 This is usually called *transformer architecture*.



(a) Multi-Head Self-Attention (Slide 31)

(b) Full Transformer Layer (Slide 42)

Figure 2: Official Lecture Diagrams: Multi-Head Self-Attention and Full Transformer Architecture.

Perspective 2: Official Lecture Multi-Head Self-Attention & Layer Architecture (COMP0264 Lecture 2, Slides 31 & 42) Self-attention maps input sequence $X \in \mathbb{R}^{N \times d_{\text{model}}}$ to Queries (Q), Keys (K), and Values (V):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Attention output matrix:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- Dynamic Content Routing:** Computes pairwise similarity between all token pairs directly ($O(1)$ path length), eliminating vanishing gradients over long sequences.
- Parallel Training:** Unlike RNNs ($h_t = f(h_{t-1}, x_t)$ which require sequential processing), matrix product QK^T processes all N tokens simultaneously on GPUs.

Part (b): Positional Encoding Necessity & Formulations

Core Concept Summary: Self-attention is permutation equivariant ($P\text{Attn}(X) = \text{Attn}(PX)$), requiring explicit positional encodings (sinusoidal or RoPE 2D rotation matrices) to inject word order information.

1. Sinusoidal Positional Encoding (Vaswani et al. 2017):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Added directly to word embeddings: $E_{\text{input}} = X_{\text{embed}} + PE$. Allows model to attend by relative positions since PE_{pos+k} can be represented as a linear function of PE_{pos} .

2. **Rotary Position Embedding (RoPE):** Applies a 2D rotation matrix to query and key vectors in complex plane:

$$R_{\Theta,m}^d q_m = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 \\ \sin m\theta_1 & \cos m\theta_1 \end{pmatrix} \begin{pmatrix} q_{m,1} \\ q_{m,2} \end{pmatrix}$$

Inner product $(R_{\Theta,m} q_m)^T (R_{\Theta,n} k_n) = q_m^T R_{\Theta,n-m} k_n$ depends purely on relative distance $n - m$.

Part (c): Mathematical Justification of Scaling Factor $\sqrt{d_k}$

Core Concept Summary: Unscaled dot products $q \cdot k$ have variance d_k , pushing softmax inputs into vanishing gradient regions ($\frac{\partial \sigma}{\partial z} \approx 0$). Scaling by $\sqrt{d_k}$ rescales variance to 1.

Assume components of query $q \in \mathbb{R}^{d_k}$ and key $k \in \mathbb{R}^{d_k}$ are independent random variables with zero mean ($\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$) and unit variance ($\text{Var}(q_i) = \text{Var}(k_i) = 1$).

Consider the dot product scalar $S = q \cdot k = \sum_{i=1}^{d_k} q_i k_i$:

1. **Expected Value Calculation:**

$$\mathbb{E}[S] = \mathbb{E}\left[\sum_{i=1}^{d_k} q_i k_i\right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$$

2. **Variance Derivation:**

$$\text{Var}(S) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i)$$

For independent zero-mean variables: $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] - 0 = (1)(1) = 1$.

$$\text{Var}(S) = \sum_{i=1}^{d_k} 1 = d_k$$

Standard deviation of dot product S is $\sqrt{\text{Var}(S)} = \sqrt{d_k}$.

3. **Gradient Saturation & Rescaling Justification:** For large projection dimensions d_k (e.g. $d_k = 64$ or 128), unscaled dot products have large variance (d_k), pushing inputs into extreme tails of the softmax function. In these regions, softmax outputs approach 1 or 0, and its derivative vanishes ($\frac{\partial \text{softmax}(z_i)}{\partial z_j} \approx 0$). Dividing by $\sqrt{d_k}$ rescales $\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{\text{Var}(q \cdot k)}{d_k} = \frac{d_k}{d_k} = 1$, keeping gradients well-behaved.

Solution 3: Large Language Models (LLMs)

Part (a): Emergent In-Context Learning (ICL)

Core Concept Summary: In-Context Learning (ICL) is probabilistic implicit Bayesian inference over latent task variable $\tau \sim P(\tau)$, where prompt examples update posterior $P(\tau | C)$ without updating weights.

In-Context Learning (ICL) is modeled probabilistically as **implicit Bayesian inference** over a latent task variable $\tau \sim P(\tau)$. During pretraining on large text corpora, next-token prediction minimizes:

$$\mathcal{L} = - \sum_t \log P(x_t | x_{<t})$$

Since training documents contain diverse tasks, the marginal likelihood integrates over latent tasks:

$$P(x_1, \dots, x_N) = \int P(\tau) \prod_{t=1}^N P(x_t | x_{<t}, \tau) d\tau$$

When an LLM is presented with prompt tokens $C = (x_1, y_1, x_2, y_2, \dots, x_k)$, evaluating $P(y_k | C, x_k)$ implicitly computes the posterior task distribution $P(\tau | C)$ via Bayes' rule:

$$P(\tau | C) \propto P(\tau) P(C | \tau)$$

The model uses prompt examples C as evidence to locate the task vector τ , allowing it to execute new tasks without parameter updates.

Part (b): Masked Self-Attention & Autoregressive Property

Core Concept Summary: Causal masking adds matrix $M_{ij} = -\infty$ for $j > i$ into attention logits, enforcing autoregressive factorization $p(x_1 \dots x_T) = \prod p(x_t | x_{<t})$ during parallel training.

Perspective 1: Matrix Causal Mask Grid Diagram (TikZ Diagram)

Causal Mask Matrix M_{ij} ($N = 4$)

	Tok 1	Tok 2	Tok 3	Tok 4
Token 1	0	$-\infty$	$-\infty$	$-\infty$
Token 2	0	0	$-\infty$	$-\infty$
Token 3	0	0	0	$-\infty$
Token 4	0	0	0	0

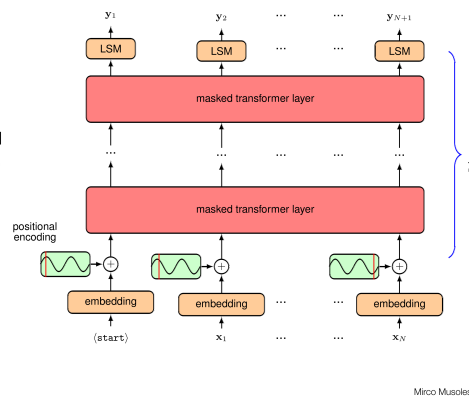
Green (0): Allowed causal attention
 Red ($-\infty$): Masked out (softmax $\rightarrow$ 0)
 Preserves autoregressive factorization $p(x_t | x_{<t})$

Training a GPT Model

- We therefore introduce mask attention (sometimes called causal attention), into each the attention layers.
- The modification is simple:
 - We set to zero all the attention weights that correspond to a token attending to any token later in the sequence.
 - In other words, we set all the corresponding elements of the attention matrix.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}[\mathbf{Q}\mathbf{K}^T]\mathbf{V}$$

to 0 and we normalise the remaining elements so that each row sums to 1.



Generative AI 2025-26

Mirco Musolesi

Figure 3: *Official Lecture Diagram: Autoregressive GPT Training & Causal Masking.*

Perspective 2: Official Lecture Architecture Diagram (COMP0264 Lecture 3, Slide 31)

The joint probability of a sequence factorizes autoregressively: $p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t})$. To train the entire sequence in parallel without leaking future tokens $x_{>t}$ into x_t , a **causal mask matrix** $M \in \mathbb{R}^{T \times T}$ is added to attention logits before softmax:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Since $\exp(-\infty) = 0$, attention weights for future positions $j > i$ are zeroed out, ensuring x_t depends only on $x_{\leq t}$.

Part (c): Factual Correctness vs MLE & Solutions

Core Concept Summary: MLE minimizes mode-covering $KL(P_{\text{data}} \parallel P_\theta)$, prioritizing surface n-gram fluency over truth. Preference alignment (DPO/RLHF) and RAG enforce factual precision.

1. **Why Maximum Likelihood (MLE) Fails for Factual Correctness:** MLE minimizes $KL(P_{\text{data}} \| P_{\theta}) = \mathbb{E}_{P_{\text{data}}}[\log \frac{P_{\text{data}}}{P_{\theta}}]$. Because KL is ****mode-covering**** (zero-avoiding for P_{data}), the model is penalized heavily for assigning zero probability to data sequences, encouraging it to assign non-zero probability to plausible-sounding hallucinations that match superficial n-gram statistics. MLE optimizes fluency, not truth.
2. **Solutions for Factual Alignment:**
 - **Direct Preference Optimization (DPO) / RLHF:** Optimizes preference loss on pair (y_w, y_l) (winning vs losing response):

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Penalizes ungrounded hallucinations y_l .

- **Retrieval-Augmented Generation (RAG):** Conditions generation on verified external knowledge documents R : $P(y | x, R)$.

Part (d): Catastrophic Forgetting & KL Divergence

Core Concept Summary: Catastrophic forgetting corresponds to unbounded parameter drift $KL(P_{\theta_{\text{pre}}} \| P_{\theta_{\text{ft}}}) \rightarrow \infty$. KL regularization penalties preserve pre-trained capabilities.

Fine-tuning updates parameters from pre-trained state θ_{pre} to θ_{ft} on task distribution P_{target} . From a probabilistic perspective, catastrophic forgetting occurs when probability mass assigned to general knowledge distribution $P_{\text{pre}}(y | x)$ drops significantly:

$$KL(P_{\theta_{\text{pre}}}(y | x) \| P_{\theta_{\text{ft}}}(y | x)) \rightarrow \infty$$

To prevent parameter drift away from pre-trained capabilities, fine-tuning objectives add an explicit KL regularization penalty:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{target}}(\theta) + \beta KL(P_{\theta}(y | x) \| P_{\theta_{\text{ref}}}(y | x))$$

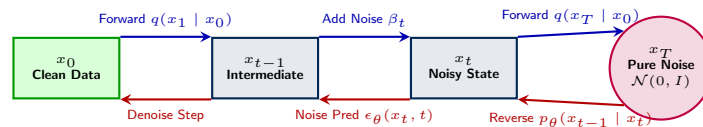
where $P_{\theta_{\text{ref}}}$ is the frozen pre-trained model reference.

Solution 4: Diffusion Models

Part (a): Forward and Reverse Process Mathematical Formulation

Core Concept Summary: Diffusion models define a forward Markov noising chain $q(x_t | x_0) = \mathcal{N}(\sqrt{\alpha_t}x_0, (1 - \bar{\alpha}_t)I)$ and train a reverse decoder $p_{\theta}(x_{t-1} | x_t)$ via score matching.

Perspective 1: Discrete Markov Chain Pipeline (TikZ Diagram)



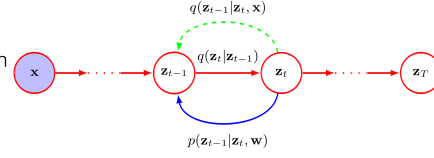
Perspective 2: Official Lecture Architecture Diagram (COMP0264 Lecture 5, Slide 20)

1. **Forward (Noising) Process $q(x_{1:T} | x_0)$:** A fixed Markov chain that adds Gaussian noise according to variance schedule $\beta_1, \dots, \beta_T$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

Reverse Decoder

- We have seen that the forward encoder model is defined by a sequence of Gaussian conditional distributions $q(\mathbf{z}_t | \mathbf{z}_{t-1})$.
- However, inverting this directly leads to a distribution $q(\mathbf{z}_{t-1} | \mathbf{z}_t)$ that is intractable.
- In fact, it would require to integrate over all the possible values of the starting vector $\mathbf{x}$.
- But we cannot “sample” $\mathbf{x}$, since that is the vector we want to model in the first place.



Generative AI 2025-26

Mirco Muscolesi

Figure 4: *Official Lecture Diagram: Diffusion Model Forward & Reverse Processes.*

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Using the reparameterization trick repeatedly:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1} = \dots = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$. Thus the marginal distribution at step t is closed-form:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$$

2. **Reverse (Denoising) Process** $p_\theta(x_{0:T})$: Starts from Gaussian noise $p(x_T) = \mathcal{N}(x_T; 0, I)$ and parameterizes transitions:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

When conditioned on x_0 , the true posterior $q(x_{t-1} | x_t, x_0)$ is tractable Gaussian via Bayes' rule:

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0)q(x_{t-1} | x_0)}{q(x_t | x_0)} = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

where:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

Part (b): Reverse Decoder Training & ϵ -Prediction Objective

Core Concept Summary: DDPM trains neural network $\epsilon_\theta(x_t, t)$ to predict added noise ϵ using simplified MSE loss $\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$.

Direct ELBO training minimizes $KL(q(x_{t-1} | x_t, x_0) \parallel p_\theta(x_{t-1} | x_t))$. Since $x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$, substituting x_0 into posterior mean $\tilde{\mu}_t$ gives:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

This motivates parameterizing model mean $\mu_\theta(x_t, t)$ with a neural network $\epsilon_\theta(x_t, t)$ predicting noise ϵ :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

Simplifying the weighted KL terms yields the ****DDPM Mean Squared Error (MSE) Loss**** (Ho et al. 2020):

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim U(1, T), x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0, I)} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\alpha_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t \right) \right\|^2 \right]$$

APPENDIX: Standard Formulas & Foundational Identities

1. Chain Rule of Probability

The Chain Rule decomposes any joint probability distribution over multiple variables into a product of conditional probabilities.

2 Variables ($N = 2$):

$$p(x_1, x_2) = p(x_1)p(x_2 | x_1) = p(x_2)p(x_1 | x_2)$$

3 Variables ($N = 3$):

$$p(x_1, x_2, x_3) = p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2)$$

General N Variables ($N \geq 2$):

$$p(x_1, x_2, \dots, x_N) = p(x_1) \prod_{i=2}^N p(x_i | x_1, x_2, \dots, x_{i-1})$$

2. Bayes' Rule & Law of Total Probability

Bayes' Rule:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)} = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}$$

Law of Total Probability:

$$p(x) = \sum_y p(x, y) = \sum_y p(x | y)p(y)$$

3. Evidence Lower Bound (ELBO) & Jensen's Inequality

For any continuous variational posterior $q(z | x)$:

$$\log p(x) \geq \mathbb{E}_{q(z|x)} \left[\log \frac{p(x, z)}{q(z | x)} \right] = \mathbb{E}_{q(z|x)} [\log p(x | z)] - KL(q(z | x) \parallel p(z)) \equiv \mathcal{L}_{\text{ELBO}}$$

4. Gaussian Closed-Form KL Divergence

For $q(z | x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and prior $p(z) = \mathcal{N}(0, I)$ in d latent dimensions:

$$KL(q(z | x) \parallel p(z)) = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

5. Variance Scaling in Self-Attention ($\sqrt{d_k}$)

For independent zero-mean unit-variance queries q_i and keys k_i :

$$\text{Var}(q \cdot k) = \text{Var} \left(\sum_{i=1}^{d_k} q_i k_i \right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \implies \text{Var} \left(\frac{q \cdot k}{\sqrt{d_k}} \right) = \frac{d_k}{d_k} = 1$$

6. Direct Preference Optimization (DPO) Loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$